



Habib University
shaping futures

Habib University

OBJECT-ORIENTED PROGRAMMING FALL 2025

Week 07 - Part 1

Operator Overloading





THIS WEEK'S AGENDA

- 01. Introduction to Operator Overloading
- 02. Overloading Unary Operators
- 03. Overloading Binary Operators
- 04. Assignment and Comparison
- 05. Extraction and Insertion Operators





INTRODUCTION TO OPERATOR OVERLOADING





INTRODUCTION TO OPERATOR OVERLOADING

- Recall that we have already explored function overloading in detail which entailed changing a function's signature (the number, type or order of the parameters) while keeping the same name.
- C++ also allows us to overload **operators**.
- This means that we can define our own functionality for the common operators like `+`, `-`, `*`, `/`, `++`, `--`, `<`, `>`, etc.
- These overloaded definitions can be useful when working with user-defined data types (objects of classes that we have created).





OPERATOR OVERLOADING

- Let's look at the Distance class we have been working with in the past.

```
3  class Distance {  
4      private:  
5          int feet;  
6          int inches;  
7      public:  
8          Distance(int f, int i) : feet(f), inches(i) {}  
9  };
```

- What if we want to add two Distance objects to produce a third?
Would $d_3 = d_2 + d_1$ work?





OPERATOR OVERLOADING

- The keyword **operator** is used to overload an operator. This is used in the function declaration as shown below:

Return type Operator symbol Parameters

int **operator-**(Time a, Time b)

Tasks to be performed by the overloaded operator

```
{
    return a.get_hours() * 60 + a.get_minutes()
    - b.get_hours() * 60 - b.get_minutes();
}
```





OVERLOADING UNARY OPERATORS





OVERLOADING UNARY OPERATORS

- Some of the unary operators that we have seen before are: ++, --, !
- Consider the class Counter:

```
11  class Counter {  
12      private:  
13          unsigned int count = 0;  
14      public:  
15          void incrementCount() {  
16              count++;  
17          }  
18          int getCount() {  
19              return count;  
20          }  
21  };
```





OVERLOADING UNARY OPERATORS

- Currently we are using the incrementCount() function to increment the count, but we can use the ++ operator to do that. But we need to overload it first:

```
void operator ++() {  
    ++count;  
}
```

- To make a call to this new function we do the following:

```
Counter c1;  
++c1;
```





OVERLOADING UNARY OPERATORS

- The only way the compiler can distinguish between **overloaded functions** is through the **different signatures**.
- Similarly, the way it distinguishes between **overloaded operators** is by looking at the **data type of its operands**.
- If the operand is a basic type such as an int (++intvar), then the compiler will use the built-in routine to increment the integer.
- However, if the operand is a class variable (as in the case of our Counter class), the compiler will know to use our user-written operator++() function instead.





OVERLOADING OPERATOR ++

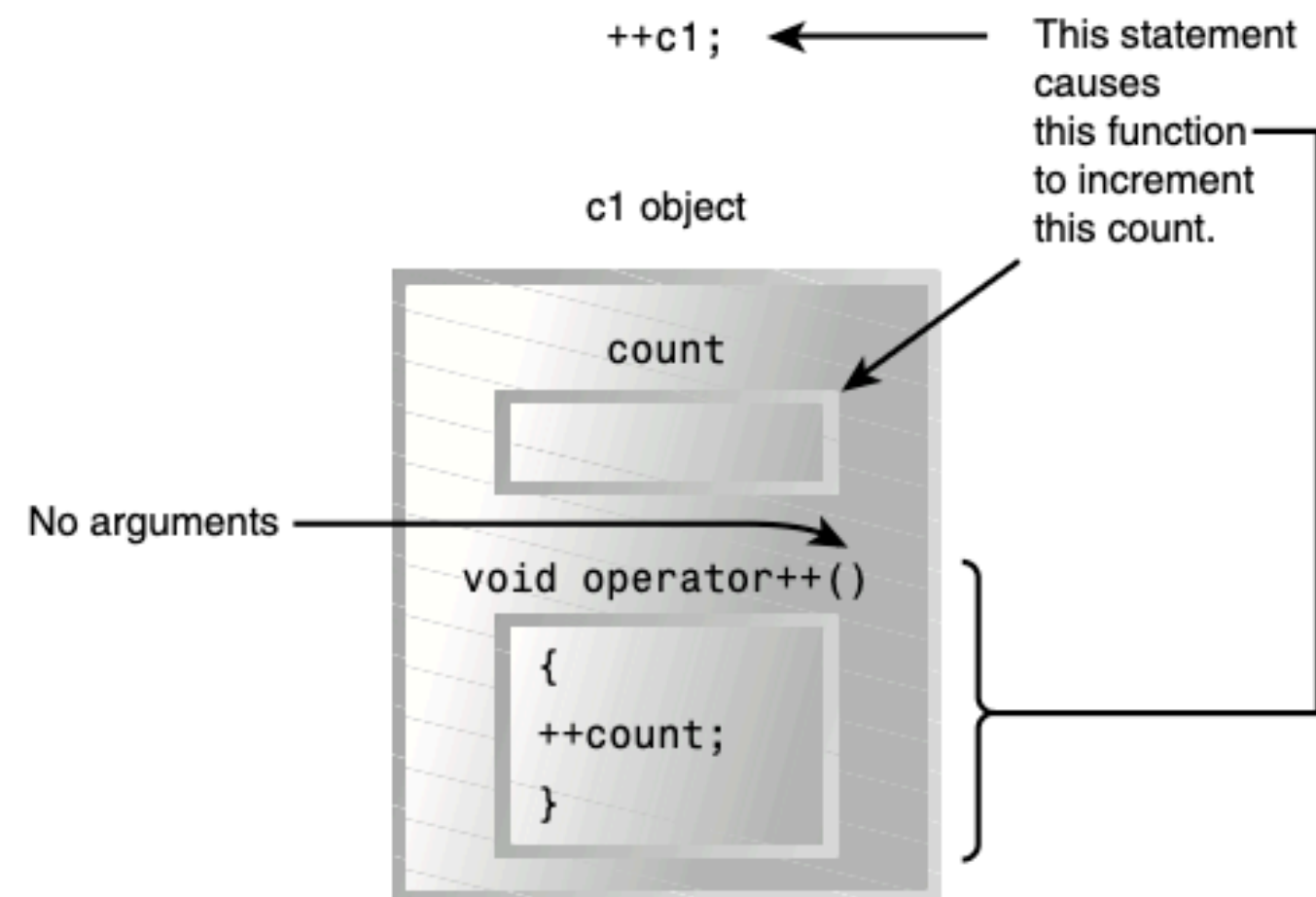


FIGURE 8.1

Overloaded unary operator: no arguments.





OPERATOR OVERLOADING - RETURN TYPE

- You can also overload an operator to return a type as follows:

```
Counter operator ++() {  
    ++count;  
    Counter temp;  
    temp.count = count;  
    return temp;  
}
```

```
Counter c1;  
Counter c2 = ++c1;
```





OVERLOADING OPERATOR ++

```
11  class Counter {  
12      private:  
13          unsigned int count = 0;  
14      public:  
15          Counter() : count(0) {}  
16          Counter(int c) : count(c) {}  
17          Counter operator ++() {  
18              ++count;  
19              return Counter(count);  
20          }  
21  };
```





OVERLOADING POSTFIX OPERATORS

- In order to overload the postfix ++ operator, we need to distinguish it from the prefix ++ operator.
- The key difference is that the postfix version takes a dummy integer parameter, whereas the prefix version does not take any parameters like we saw.
- In order to overload the postfix ++ operator:
 - Define the operator ++ function in the class as a member function.
 - Use the dummy int parameter to signal that it is the postfix version.
 - Return the object in the original state before the increment, typically by returning a copy of the current object.
- Let's look at an example:





OVERLOADING POSTFIX OPERATORS

```
Counter operator ++(int) {  
    return Counter(count++);  
}
```

```
Counter c1;  
Counter c2 = c1++;
```





PREFIX VS POSTFIX INCREMENTS

- Prefix notation:
 - Counter operator `++()`
- Postfix notation:
 - Counter operator `++(int)`
- The only difference is the int in the parentheses.
- **This isn't really an argument, and it doesn't mean integer.**
- It's just to signal to the compiler to create the postfix version of the operator.





DECREMENT OPERATORS --

- The same mechanism is applicable to the decrement unary operator as well.
- The decrement operator also has both the prefix and the postfix versions:
 - Prefix:
 - Counter operator `--()`;
 - Postfix:
 - Counter operator `--(int)`;
- **Task: Write the functions for overloading the decrement operator for the Counter class.**





OVERLOADING BINARY OPERATORS





OVERLOADING ADDITION OPERATOR

```
3  class Distance {
4      private:
5          int feet;
6          float inches;
7      public:
8          Distance(int f, float i) : feet(f), inches(i) {}
9          Distance operator +(Distance &d) const {
10             int f = feet + d.feet;
11             float i = inches + d.inches;
12             while (i >= 12.0) {
13                 i -= 12.0;
14                 f++;
15             }
16             return Distance(f, i);
17         }
18 };
```





OVERLOADING ADDITION OPERATOR

```
24  int main() {  
25      Distance d1, d2, d3;  
26      Distance d4(11, 6.25);  
27      Distance d5(5, 15.75);  
28      d3 = d4 + d5;  
29      d2 = d1 + d4 + d5;  
30      std::cout << "D1: "; d1.getDistance();  
31      std::cout << "D2: "; d2.getDistance();  
32      std::cout << "D3: "; d3.getDistance();  
33      std::cout << "D4: "; d4.getDistance();  
34      std::cout << "D5: "; d5.getDistance();  
35      return 0;  
36  }
```





BINARY OPERATOR OVERLOADING

- The same mechanism is applicable to subtraction, multiplication, and division operators.
- Note - to add two distances, we can make the following function:
 - `Distance operator +(Distance &d1, Distance &d2);`
- This declaration states that d1 and d2 will be added and the resulting distance will be returned.
- This function can also be declared as:
 - `Distance operator +(Distance &d2);`
- Here d1 is an implicit operator and both types of functions can be called with:
 - `d3 = d1 + d2;`



BINARY OPERATOR OVERLOADING

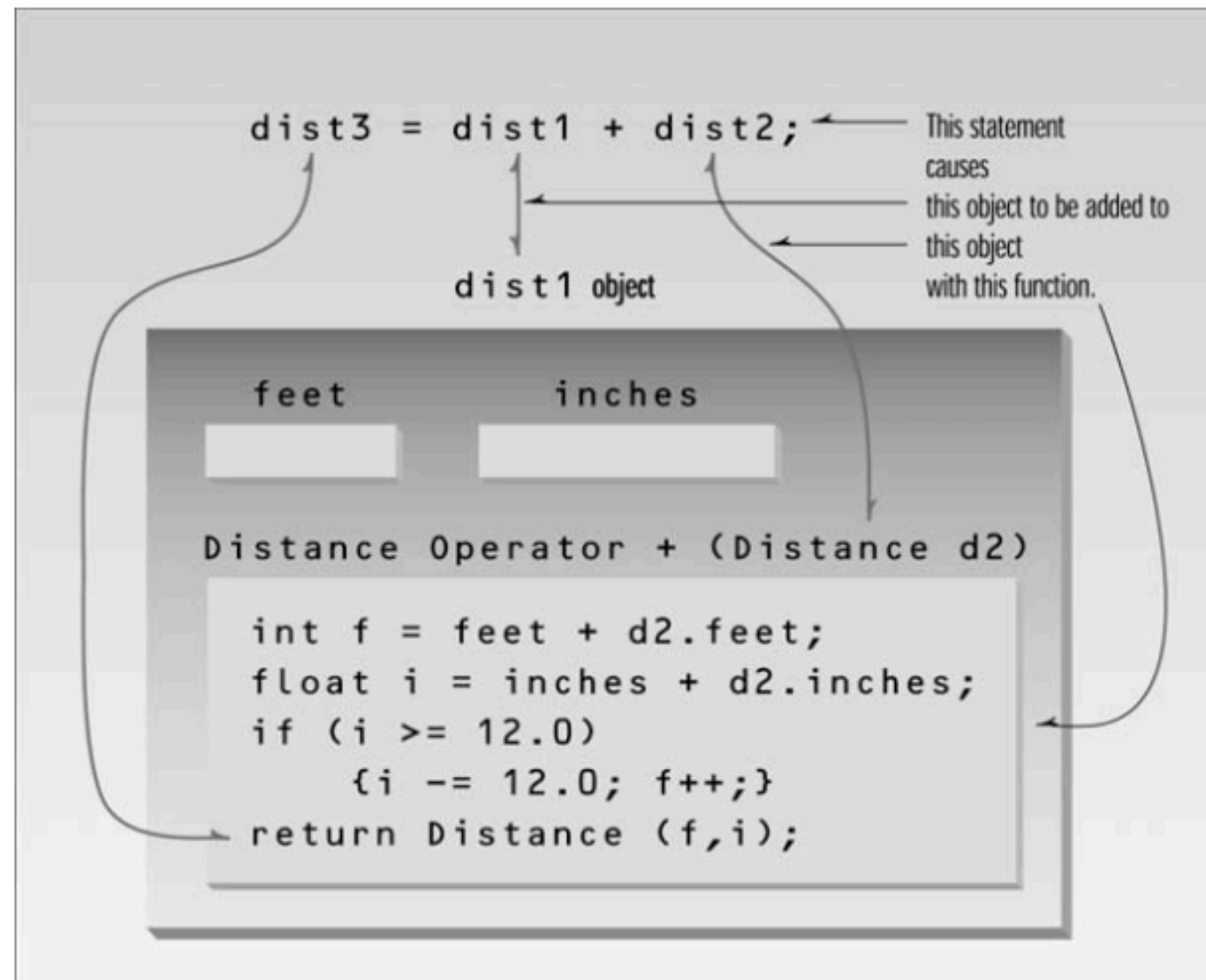


FIGURE 8.2

Overloaded binary operator: one argument.



BINARY OPERATOR OVERLOADING

- For the operator `+`() function, the left operand is accessed directly using `feet` and `inches`, since this is the object of which the operator is a member.
- The right operand is accessed as the function's argument, as `d2.feet` and `d2.inches`.
- We can form a generalization here:
 - An overloaded operator always requires one less argument than its number of operands, since one operand is the object of which the operator is a member.
- This is why unary operators require no arguments.





BINARY OPERATOR OVERLOADING

- All other binary operators can also be overloaded but there are some rules.
 1. **Almost any operator that exists can be overloaded.**
 2. You must **only overload operators that exist.**
 3. **At least one of the arguments in an overloaded operator must be a user-defined type.**
 4. It is **not possible** to change the number of operands that an operator supports.
 5. The **precedence rules** (associativity, precedence) between operators **cannot** be changed.





ASSIGNMENT AND COMPARISON





COMPARISON OPERATORS

- It is also possible to overload the ==, !=, and other comparison operators. For example:

```
bool operator==(const Distance &other) const {  
    return feet == other.feet && inches == other.inches;  
}  
  
bool operator!=(const Distance &other) const {  
    return !(*this == other);  
}
```





EXTRACTION AND INSERTION OPERATORS





EXTRACTION AND INSERTION OPERATORS

- Here ostream represents the output stream, and the function accepts an output stream as a parameter along with a user-defined class object to be sent to the output stream.
- It then returns the output stream by reference with the user-defined object printed.
- Let's look at an example:

```
21  std::ostream& operator <<(std::ostream& out, Distance &d) {  
22      out << "Feet: " << d.getFeet() << ", Inches: " << d.getInches();  
23      return out;  
24  }
```





EXTRACTION AND INSERTION OPERATORS

- We can make use of this overloaded extraction operator in the following way:
 - `Distance d1(5, 5.12);`
 - `std::cout << d1 << std::endl;`
- What this really means is:
 - `(std::cout << d1) << std::endl;`
- And the function is called with these arguments:
 - `operator <<(cout, d1);`

```
21  std::ostream& operator <<(std::ostream& out, Distance &d) {  
22      out << "Feet: " << d.getFeet() << ", Inches: " << d.getInches();  
23      return out;  
24  }
```





EXTRACTION AND INSERTION OPERATORS

```
26  std::istream& operator >>(std::istream &in, Distance &d) {  
27      int feet;  
28      double inches;  
29      std::cout << "Enter the feet and inches: ";  
30      in >> feet >> inches;  
31      d = Distance(feet, inches);  
32      return in;  
33  }
```

```
Distance d1;  
std::cin >> d1;
```





SUMMARY





SUMMARY

- To overload an operator, we use the operator keyword followed by the operator itself.
- There are certain rules and restriction that are applicable to operator overloading which we saw in detail during the lecture.
- Overall, operator overloading is a very useful feature in C++.





READINGS

- Object-Oriented Programming in C++ by Robert Lafore - Chapter 08





ACKNOWLEDGEMENTS

Many of the slides of this lecture have been taken/modified from the lecture notes of:

- Object Oriented Programming Fall 2024 by Dr. Faisal Alvi, Assistant Professor, Computer Science
- Object Oriented Programming Fall 2023 by Dr. Musabbir Majeed, Assistant Professor, Computer Science

